

FSM steering-loop findings

anyloop tip/tilt loop — measurement session 2026-07-10

Executive summary

The loop was not gain-limited — it was crippled by three independent, now-measured defects. With those fixed, both plant axes are flat gain + 1.4 ms pure delay with no resonance below 300 Hz, and an integrator-only controller at ~41 Hz crossover is well supported.

1. Serial command pacing (DAC path)

piplate_bridge throttles writes by the configured baud, but /dev/ttyACM0 is USB CDC — the baud does not set the wire speed, only the pacing. At the default 115200 the actuator gets at most ~440 commands/s (~2.3 ms hold on a 2310 Hz loop). Raising to 921600 overruns the bridge's real drain rate (~1800 cmd/s) and builds a constant ~10 ms buffer queue. The sweet spot is 460800: ~1770 cmd/s pacing, latency stays at the physical 1.4 ms. See page 3.

2. Geometry: the setpoint was unreachable

The auto-centred 64×64 ROI put the beam 4 px from the top edge, ~28 px from the frame-centre setpoint in y. The fine stages have only ±13 px (x) / ±9 px (y) of authority — measured from the step gains — so both integrators wound to the rail and sat there; more gain only made the limit-cycling worse. Fixed with start_x=920 / start_y=488 (ROI centred on the beam's parked position, sensor ~(y 520, x 952) at the current DAC biases).

3. Coarse channels live with unverified sign

steering_tuned.json had ch2/ch3 at scale ±2 driven by the raw error — against the config's own push-test rule; a wrong-signed coarse drive is positive feedback. Parked at bias (0.8 / 2.394 V) until sign-verified.

Verified along the way

- Axis map, by direct step test: CoM element 0 = y = DAC ch0 (+2 V/unit); element 1 = x = DAC ch1 (−2 V/unit). Positive integrator gain is negative feedback on both axes.
- End-to-end latency = 1.1 ms fixed + 1.4 frame periods (page 2); exposure is irrelevant.
- All pre-2026-07 Bode data is invalid: different DAC step sizes, serial bufferbloat, an edge-clipped beam, and reflection-diluted whole-frame centroids each corrupted a run.
- The stray reflection sits INSIDE the new centred ROI at ~(10, 28); the loop must use the tracked window initialised on the beam (init_y/init_x = 32), not an acquisition phase.
- latency_test device reworked: drift-immune calibration (step from consecutive window diffs, noise from within-window scatter), per-edge re-baselining, parked output on all error paths. Doubles as a step-gain and axis-mapping probe.

Controller (applied to steering_tuned.json)

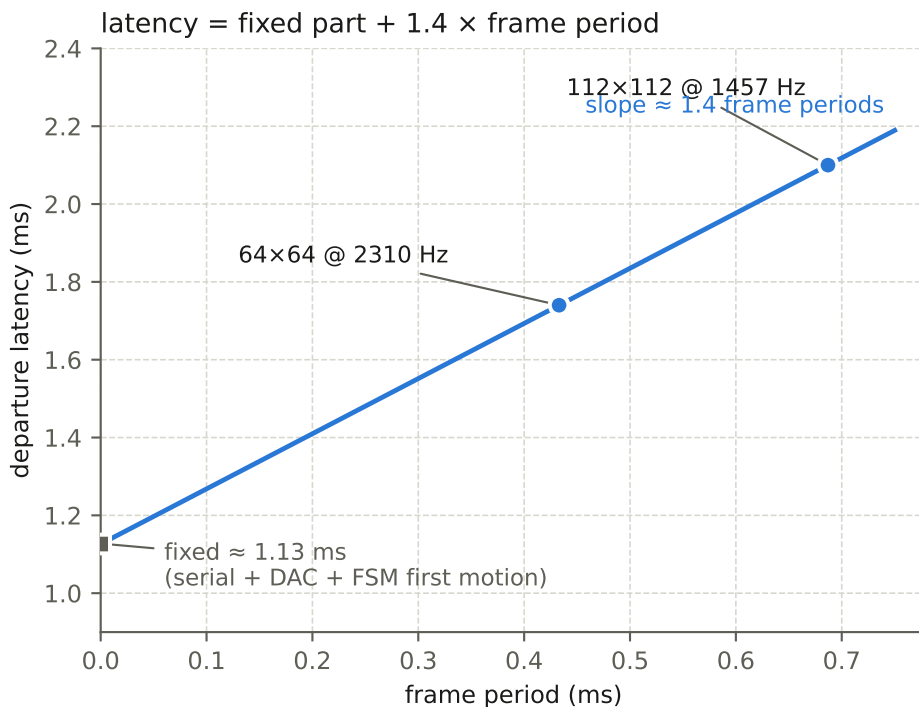
Integrator-only. i = 600 (x), iy = 850 (y) → both axes ~41 Hz crossover, ~60° phase margin at the effective 2.0 ms delay (1.4 ms path + 0.6 ms command hold at baud 460800). Headroom to i = 900 / iy = 1300 (~60 Hz, 45°). No biquad/notch: there is nothing to notch. Page 5.

Latency: measured, decomposed

latency_test steps one DAC channel and timestamps the command edge and the first frame whose centroid moved: the full path serial→DAC→FSM→exposure→readout/USB→centroid. 40 edges per run, 0 missed in every run. Two frame rates and two exposures separate the contributions.

Run matrix (departure = first motion; 50% = half the step height)

ROI	exposure	loop rate	departure (median)	50% crossing	frames
112×112	330 μs	1457 Hz	2.10 ms	2.59 ms	3.0
112×112	100 μs	1457 Hz	2.06 ms	2.20 ms	3.0
64×64 (x / ch1)	330 μs	2310 Hz	1.74 ms	2.04 ms	4.0
64×64 (y / ch0)	330 μs	2310 Hz	1.77 ms	1.77 ms	4.1

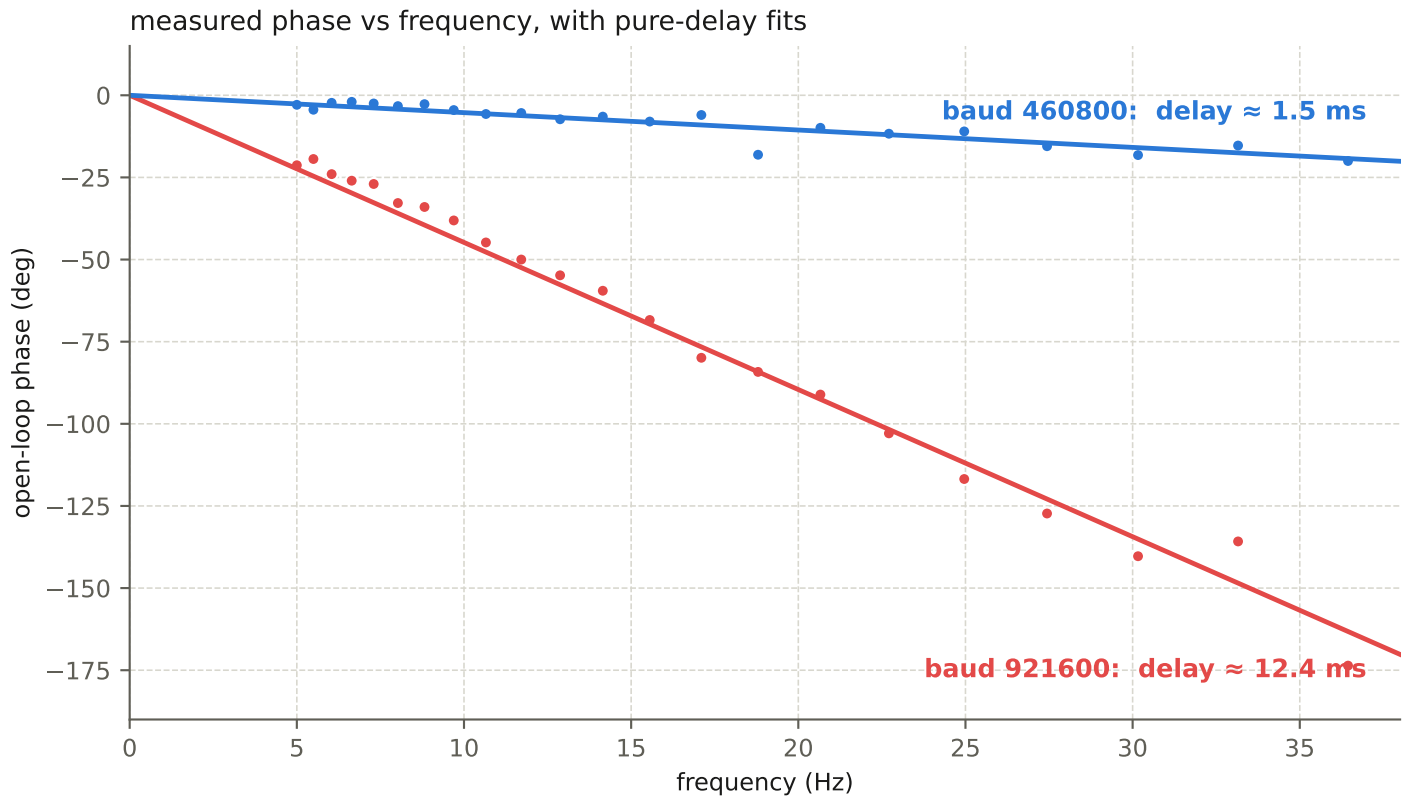


Reading it:

- The frame-proportional part (~1.4 frames) is the camera pipeline: readout/USB transfer plus the wait for the response to land in a frame. It shrinks with ROI — 64×64 buys 0.36 ms.
- Cutting exposure 330→100 μs changed nothing (2.10→2.06 ms, within jitter): integration time is not where the latency lives, and the frame rate was already readout-limited.
- The ~1.1 ms fixed part is the DAC serial transaction, FSM first motion, and fixed camera/USB latency. It is the floor that further ROI shrinking converges to — and the only part a faster command path (e.g. a different DAC) could remove.
- Latencies cluster at exact frame boundaries (e.g. 2.05/2.75 ms at 1457 Hz): the response arrives quantised to frames, ±1 frame of phase jitter.

Serial pacing: baud, throughput, and bufferbloat

Two x-axis Bode sweeps in identical geometry, differing only in the `piplate_bridge` baud setting. A pure transport delay appears as phase falling linearly with frequency; the slope IS the delay. At 921600 the command stream overruns the bridge's drain rate and a buffer somewhere in the USB/MCU path sits permanently full: a constant ~10 ms queue on every command.



What each baud setting actually does (USB CDC: baud = pacing only)

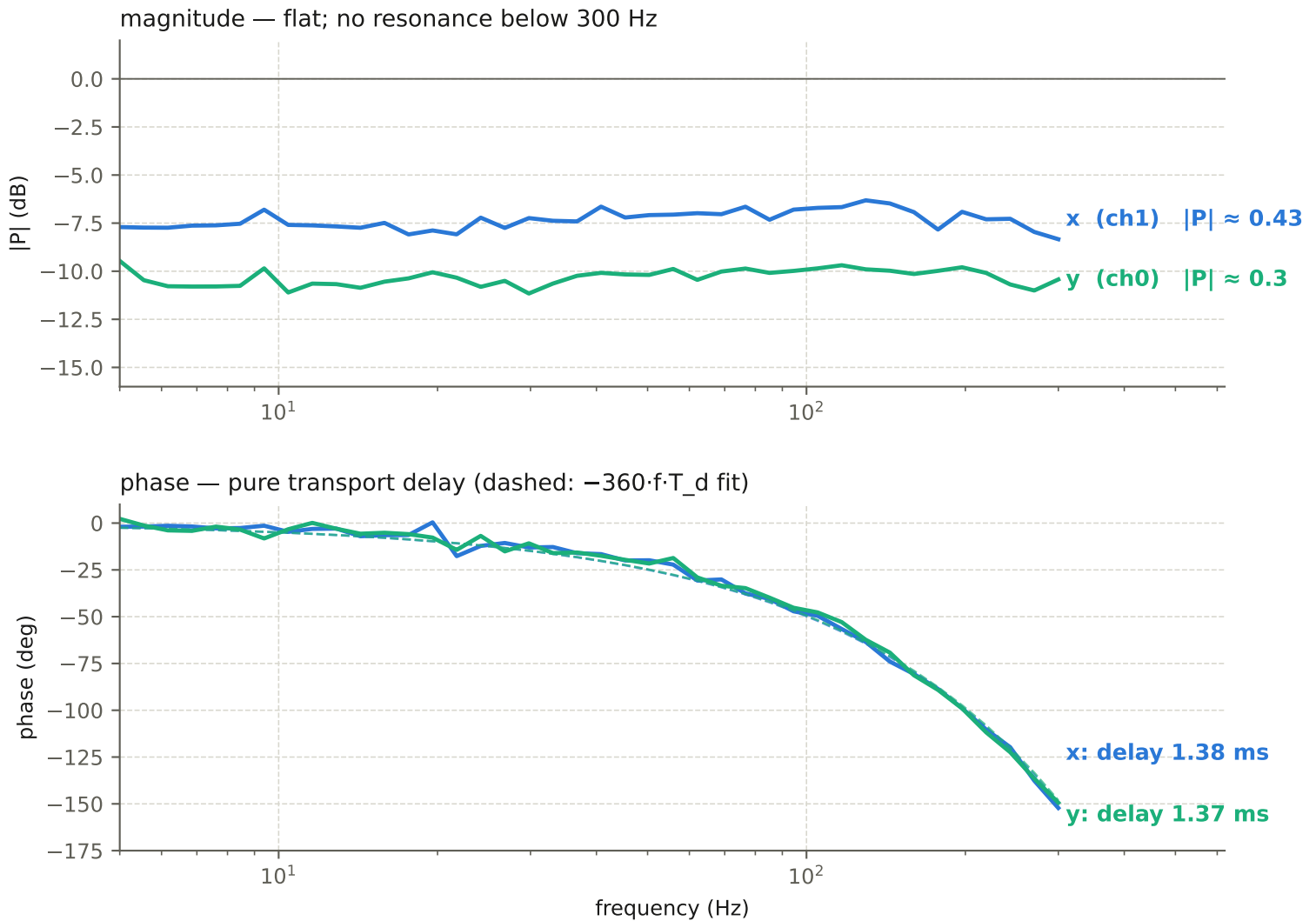
baud	pacing	behaviour	effective delay added
115200	~440 cmd/s	2.3 ms hold; two live channels $\rightarrow$ ~220/s each	+1-2.3 ms (hold)
460800	~1770 cmd/s	just under the ~1800 cmd/s drain — no queue	+~0.6 ms (hold)
921600	~3500 cmd/s	overruns drain $\rightarrow$ FIFO sits full (bufferbloat)	+~10 ms (queue)

Consequences and the change applied:

- The closed loop had been running at the default 115200: with both channels updating, each actuator refreshed at ~220 Hz — a ~2.3 ms average hold stacked on the 1.4 ms path delay, roughly tripling the effective delay the controller fought. This alone moves the stable crossover ceiling from ~60 Hz down to ~22 Hz; gains chosen for the fast plant then pumped the 40-100 Hz band, making the error visibly worse.
- `steering_tuned.json` and both Bode sweep configs now set "baud": 460800. Measured effect: open-loop phase collapsed from an 11 ms slope to the physical 1.45 ms (plot above), and the bridge sustained ~1130 writes/s with zero queueing during the sweeps.
- Do not raise past 460800: the 11 ms at 921600 was measured, not theoretical. If more command rate is ever needed, the bridge firmware/protocol is the bottleneck, not the baud.

Plant Bode, current geometry (2026-07-10, definitive)

Swept 5–300 Hz through the exact sensor path the loop uses: 64×64 ROI centred on the beam, tracked 25 px CoM window initialised on the beam, 2310 Hz loop, baud 460800. Software injection replaces one CoM element; the DAC writes every iteration.



Why every earlier sweep disagreed — each was corrupted by a different mechanism, all now identified:

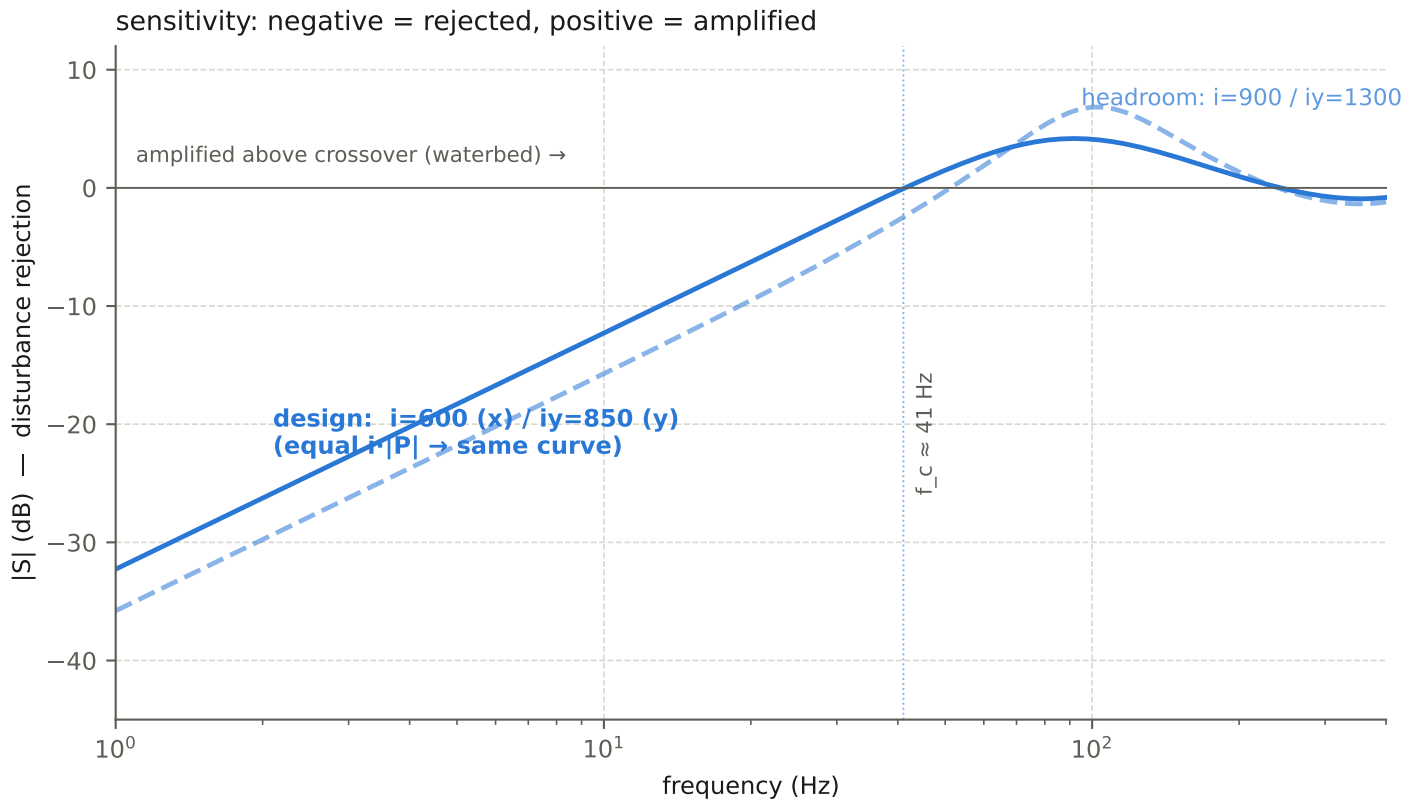
- pre-07 ("x 120 Hz / y 80–94 Hz resonances"): different DAC step sizes; numbers not comparable.
- 921600 sweep: ~10 ms bufferbloat — phase garbage (page 3), magnitudes rippled by queue jitter.
- auto-centred ROI: beam 4 px from the top edge — y response clipped and rectified to noise.
- centred ROI + whole-frame CoM: stray reflection in-frame at (10, 28) diluted x by ~2.6×.

The tracked-window sweep above is free of all four and matches the independent step-test gains.

Implication: there is nothing to notch. The old biquad/notch plan targeted resonances that do not exist in the current plant; bandwidth is limited purely by delay.

Controller design and expected rejection

Integrator-only ($p = d = 0$). With P flat and delay T_d , the open loop is $L = i \cdot P / (2\pi f) \cdot e^{(-j2\pi f \cdot T_d)}$; crossover $f_c = i \cdot P / 2\pi$ and phase margin $= 90^\circ - 360^\circ \cdot f_c \cdot T_d$. Effective $T_d = 2.0$ ms: 1.4 ms path + ~ 0.6 ms command hold at baud 460800 with both channels live.



Applied gains and what they buy

axis	gain	crossover	phase margin	rejection 4 Hz	rejection 20 Hz
x	$i = 600$	41 Hz	$\sim 60^\circ$	$\sim 10\times$	$\sim 2\times$
y	$iy = 850$	41 Hz	$\sim 60^\circ$	$\sim 10\times$	$\sim 2\times$
x (headroom)	$i = 900$	62 Hz	$\sim 45^\circ$	$\sim 15\times$	$\sim 3\times$
y (headroom)	$iy = 1300$	62 Hz	$\sim 45^\circ$	$\sim 15\times$	$\sim 3\times$

Tuning procedure:

- Start at $i=600$ / $iy=850$. Closing the loop must reduce RMS immediately; if not, stop — the problem is configuration, not gain.
- Step toward 900/1300 while watching the 60–200 Hz band of the error PSD (watch_error_ts.jl): the waterbed hump lives there and grows as margin shrinks. Back off $2\times$ from where it lifts.
- The Bode integral makes the hump unavoidable; it only moves. If the vibration spectrum has lines in 60–200 Hz, the fix is less delay (smaller ROI / faster command path), not more gain.
- p and d stay 0: with ~ 0.6 px fast centroid noise a derivative amplifies noise, and a proportional term buys little against a pure-delay plant at these margins.

Configuration changes applied

anyloop tip/tilt loop — measurement session 2026-07-10

contrib/steering_tuned.json

- asi_source: start_x = 920, start_y = 488 — ROI centred on the beam's parked position so the frame-centre setpoint is physically reachable by the fine stages.
- center_of_mass: acquisition phase removed; tracked 15 px window initialised on the beam (init_y = init_x = 32). The stray reflection is now in-frame at $\sim(10, 28)$ with flux comparable to the beam, so whole-frame acquisition could settle between the spots.
- pid: i = 600, iy = 850 (p = d = 0), start_delay 2 s (was 10 s — no acquisition to wait for).
- pplate_bridge: baud = 460800; coarse channels ch2/ch3 parked (scale 0) at 0.8 / 2.394 V per the push-test rule — they had been live at ± 2 with unverified sign; fine start_delay 2 s.

contrib/conf_bode_x.json / conf_bode_y.json (rewritten for the current setup)

- 64×64 @ 2310 Hz with the beam-centred ROI, matching the loop geometry exactly.
- Tracked 25 px CoM window with init on the beam, no acquisition phase (bode_plot injects from the first frame; the reflection must never enter the sum).
- Software injection on the correct channels for the verified axis map (x→ch1, y→ch0).
- baud 460800. Outputs: fsm2310_bode_x.pdf/.dat, fsm2310_bode_y.pdf/.dat.

contrib/conf_latency_test.json + devices/latency_test.c (new device, reworked today)

- Calibration is drift-immune: step size from consecutive high/low settled-window differences, noise from within-window scatter only; refuses to run if step < 8σ .
- Each measured edge re-baselines on the settled level of the half-cycle it departs from.
- All fatal paths park the DAC command at bias and publish it (downstream stages previously received the sensor vector and errored).
- Command step ± 0.25 (1 V on the fine channel) to clear the 8σ gate over ~ 0.6 px beam jitter.
- doc/devices/latency_test.md added.

Numbers worth remembering

- Plant: x |P| = 0.43, y |P| = 0.30 (minmax per command unit); pure delay ≈ 1.4 ms; flat to 300 Hz.
- Latency: 1.1 ms fixed + 1.4 frame periods (1.74 ms @ 2310 Hz; 2.10 ms @ 1457 Hz).
- Fine-stage authority: x ± 13 px, y ± 9 px. Beam parks at sensor $\sim(y\ 520, x\ 952)$.
- DAC bridge: drains ~ 1800 cmd/s; pace with baud 460800; never 921600 (+10 ms).
- Axis map: CoM [y, x]; y = element 0 = ch0 (+2 V/u); x = element 1 = ch1 (−2 V/u).